



PROJECT 7 · LABORATORY OF INTERNET OF THINGS

Access Control System and Surveillance with Web Cam

A QR-code-based access control system with MQTT integration, time-series logging, and optional physical door actuation.

Jacopo Veronese · 257771

THE PROBLEM

Physical keys and static badges don't scale



No easy revocation

A lost or shared physical key/badge can't be selectively invalidated without changing the whole lock.



No audit trail

Who entered, and when, is invisible unless a separate logbook is kept by hand.



Manual provisioning

Granting or removing access requires physical presence at an administration desk.

IoT offers an alternative: a short-lived, digitally-signed credential on a personal device, verified optically, logged automatically, and revocable instantly.

What Project 7 asked for



**QR-code
authentication**



Raspberry Pi camera



**MQTT as main
protocol**



**Telegram
notifications**



**Access list with
add/remove UI**



**Optional: door-lock
actuation**

Related work: why signed, short-lived QR codes

Static credentials

RFID badges, fixed QR codes

-  Cheap, no reader hardware beyond a camera
-  Photographable / clonable — the code itself never changes
-  Revoking access means reissuing or destroying the physical badge

Signed, time-limited QR

This project

-  HMAC-signed payload, single-use, 15s validity window
-  Instant revocation via an append-only access log
-  No extra reader hardware — a phone screen + a camera

Protocol choices: MQTT over HTTP polling for low-overhead publish/subscribe; InfluxDB over a relational store for natural time-series queries on access events.

Architecture overview



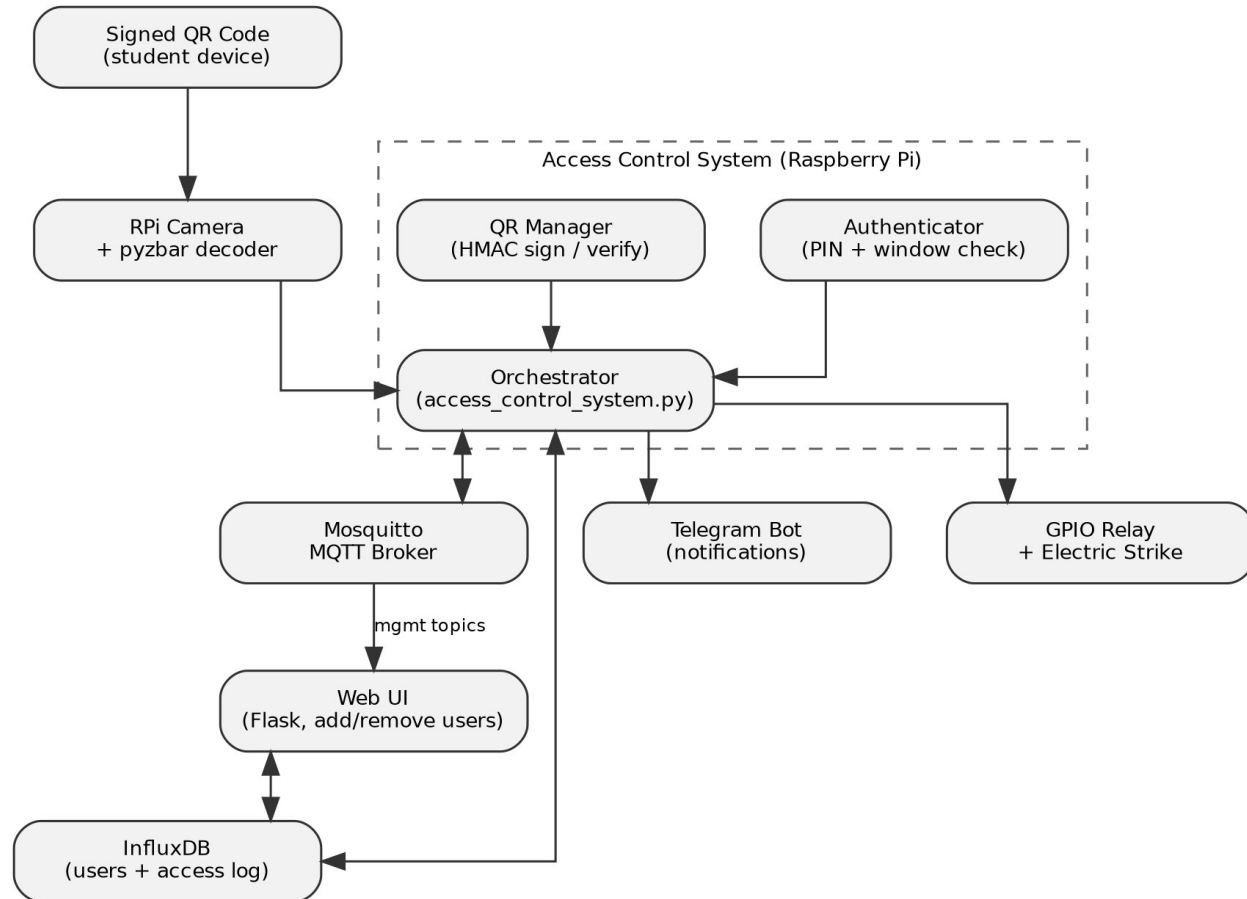
A signed QR code from the user's device is decoded by the RPi camera.



The orchestrator checks it against the Authenticator and QR Manager.

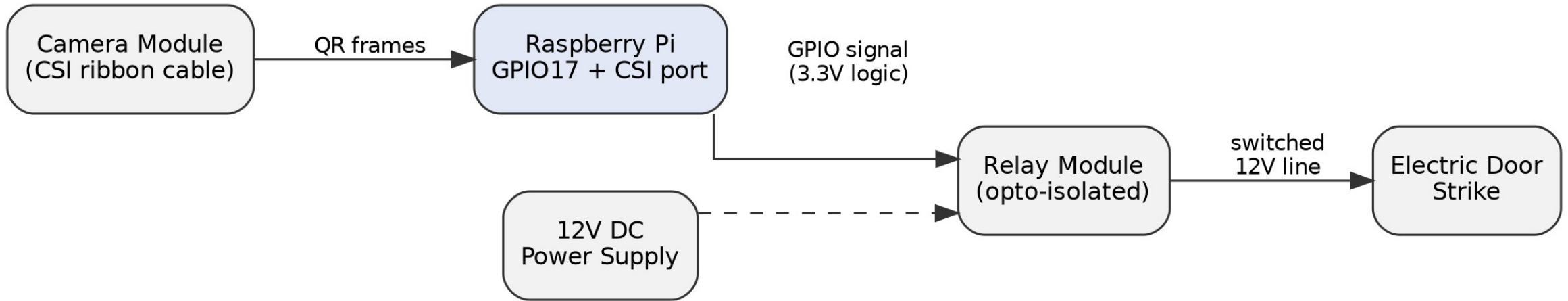


Valid access is logged to InfluxDB, announced on MQTT, and — optionally — unlocks the door and notifies Telegram.



SYSTEM DESIGN

How the physical hardware is meant to work



Scanning

The Camera Module connects to the Pi's CSI port via ribbon cable. `picamera2` continuously captures frames; `pyzbar` decodes any QR code found in each frame.



Actuation

GPIO17 drives an opto-isolated relay. The relay's low-voltage side is safely switched by the Pi's 3.3V logic; its high-voltage side switches the strike's own 12V supply — the Pi never touches mains or strike voltage directly.



Unlock pulse

On valid access, the pin goes HIGH for a configurable duration (5s default), the relay closes, the strike releases — then the pin drops LOW and the door relocks automatically.

Both paths are implemented in code (`camera_scanner.py`, `gpio_lock.py`); this diagram documents the intended wiring for the parts not yet physically validated (Section VII-C).

A QR code that can't just be photographed

1

Generate

PIN-authenticated request creates a JSON payload: user, zone, unique ID, 15s expiry.

2

Sign

HMAC-SHA256 over the payload, keyed by a server-side master secret.

3

Scan & verify

Signature checked with a constant-time comparison; code is single-use.

4

Expire

Any code older than its window is rejected, bounding replay to seconds.

IMPLEMENTATION

Technology stack



Raspberry Pi

Camera + GPIO host running the whole controller



MQTT / Mosquitto

Auth, QR, and access-list request/response API



InfluxDB

Time-series store for users and access events



Flask

Minimal web UI for managing the access list



Telegram Bot API

Notifications on granted / denied access



GPIO relay

Optional physical door-strike actuation

IMPLEMENTATION

One responsibility per module

<code>database.py</code>	InfluxDB reads/writes — users, revocation, access log
<code>authenticator.py</code>	PIN check, access-window validation, logout
<code>qr_manager.py</code>	HMAC signing / verification of QR payloads
<code>camera_scanner.py</code>	RPi camera capture + pyzbar decoding
<code>mqtt_handler.py</code>	Request/response API over MQTT
<code>gpio_lock.py</code>	Relay actuation with simulated fallback
<code>web_app.py</code>	Flask management UI

Each module is independently testable — the web UI, for example, reuses only the database layer, with no dependency on MQTT or the camera.

IMPLEMENTATION

Web management interface

- ✓ Add a user: generates a PIN and writes the grant to InfluxDB.
- ✓ Remove a user: revokes access immediately.
- ✓ Reads and writes the same data the CLI and MQTT paths use.
- ✓ Runs independently of the camera and MQTT client.

Access Control — User Management

Add user

User ID

Zone

Start

lab-1

mm / dd / yyyy , -- : -- --

End

mm / dd / yyyy , -- : -- --

Add

Added test — PIN: 328896

Active users

User ID	Zone	PIN	Start	End	
test	lab-1	328896	2026-01-25T02:00	2026-09-16T02:00	<button>Remove</button>
test1	room1	911109	2026-09-07T09:00:00	2026-09-08T18:00:00	<button>Remove</button>

Camera: available · Door lock: simulated

Real terminal output from the deployed Pi



```
$ python3 main.py
Connected to MQTT broker

=== MANAGEMENT ===
Choose option (1-6): 1
User ID: test1
Zone / room / lab (e.g. 'lab-1'): room1
User test1 added for zone 'room1'
Access window: 2026-09-07T09:00:00 -> 2026-09-08T18:00:00
PIN: 911109

=== ACCESS ===
Choose option (1-2): 1
Enter User ID: test1
Enter PIN: 911109
Temporary QR generated for test1
Valid for 15 seconds
QR saved: ./temp_qr/temp_qr_test1_20260907_231947.png
```

What's confirmed working



Verified end-to-end

- MQTT broker connectivity
- InfluxDB add / list / revoke user
- QR generation after PIN auth
- QR validation (manual test path)
- Web UI add / remove user
- Telegram delivery



Not yet validated

- Physical door-lock relay (simulated only)

A real bug, found and fixed



Symptom

A revoked-then-re-added user, under a new zone, was rejected as “not found” by authentication — even though the web UI listed them as active.



Root cause

InfluxDB groups points into separate series by full tag set. The lookup only read the first series returned — sometimes the stale, revoked one.



Fix

Scan every returned series and take the point with the latest timestamp overall — verified to resolve the discrepancy.

Limitations and future work



Diagnose the camera hardware fault and complete an end-to-end scan-to-unlock test.



Wire a physical relay + electric strike to validate `gpio_lock.py` on real hardware.



Configure a live Telegram bot and confirm notification delivery.



Add an access-log view and live status to the web UI.



Add automated tests around the multi-series lookup behavior.

CONCLUSIONS

A modular, MQTT-centric access controller

7

independent Python modules

6

MQTT request/response topic pairs

2

management interfaces (CLI + web)

Authentication, persistence, MQTT, and the web UI are implemented and verified — including a real concurrency bug found and fixed. Camera scanning and physical door actuation are implemented with safe fallbacks, awaiting hardware validation.



Thank you

Questions?

github.com/yksop/access_control